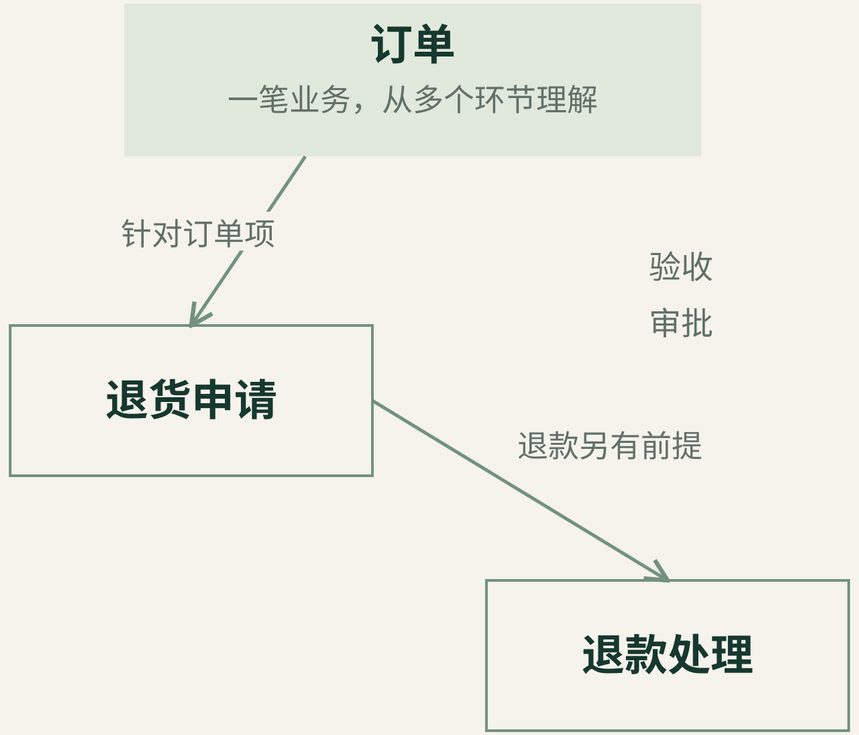
本体如何帮助 Agent 理解与处理业务



概念、系统设计与业务案例

OntoWeave（织义）
原 Ontology Platform

贯穿问题：这笔订单能退吗？为什么还不能马上退款？

目录

四章正文，最后留时间回答建设与维护中的常见问题

01 Agent 的业务理解难题 — 工具调用成功以后，为什么还会答错

02 本体的概念、技术与价值 — 业务含义怎样表达、查询与复用

03 OntoWeave 与建模方法 — 平台怎么分工，模型怎样建立

04 电商退货与退款模拟案例 — 用三笔订单核对完整判断过程

FAQ 常见问题 — 选型、建设、质量与维护

看完后，能判断何时值得用本体、Agent 怎样用、由谁建设和验证

第 1 章

Agent 的业务理解难题

01

订单已经签收，客户想退货。
Agent 应该查什么，又依据什么做判断？

任务机制 ／ 业务歧义 ／ 方法与局限

Agent 怎样完成一个业务任务

用户：订单 A101 能退吗?

01

理解请求

语言模型识别目标
拆出所需信息

02

查订单

工具返回状态、商品
以及签收时刻

03

查政策

取得适用范围
时间窗与例外

04

核对条件

调用计算工具
比较事实与条件

05

解释与继续

回答资格
说明下一步

> **语言模型**负责理解与规划。**工具**提供事实、计算或业务操作能力。
> 查到“已签收”，仍不足以判断是否符合退货条件。

调用成功说明工具完成了请求，业务判断还需要正确的含义与口径

“能退货”包含哪些业务判断

同一组事实，不同的解释会改变答案

付款

09-28 10:00

签收

10-01 10:00

拟申请

10-06 10:00

2026 年 · UTC+08:00

付款起算 **192 小时** ／ 签收起算 **120 小时**

前者超过 168 小时，后者处于时间窗内。两种解释在此均为待确认假设。

还需确认：自然日还是连续小时？普通还是定制？可申请是否等于可退款？

第一步先确认业务含义，再让 Agent 计算与回答

帮助 Agent 理解业务的六种方法

这些方法可以组合，它们补足的内容并不相同



RAG 可以检索本体，工具也可以执行已经编码的业务规则

不同方法的局限性

客服、仓库和财务，能否共同引用“申请、验收、退款”的含义?



共享文档和词典也能统一口径；是否需要本体，还要看关联与复用的价值

第 2 章

本体的概念、技术与价值

02

先说清业务里有什么、各自是什么意思、怎样关联，再讨论 Agent 如何使用。

定义与表达 ／ 使用机制 ／ 标准与适用性

什么是本体（Ontology）

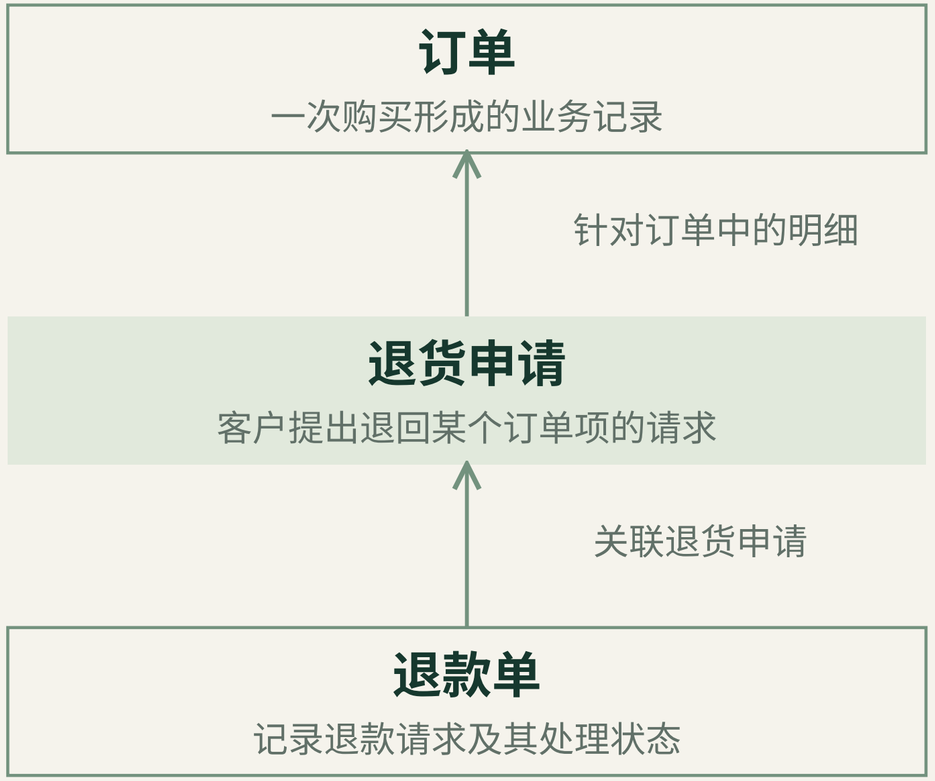
先用一个业务例子理解，再补上专业表达

把一块业务中重要的

对象、含义、关系

及适用条件整理成

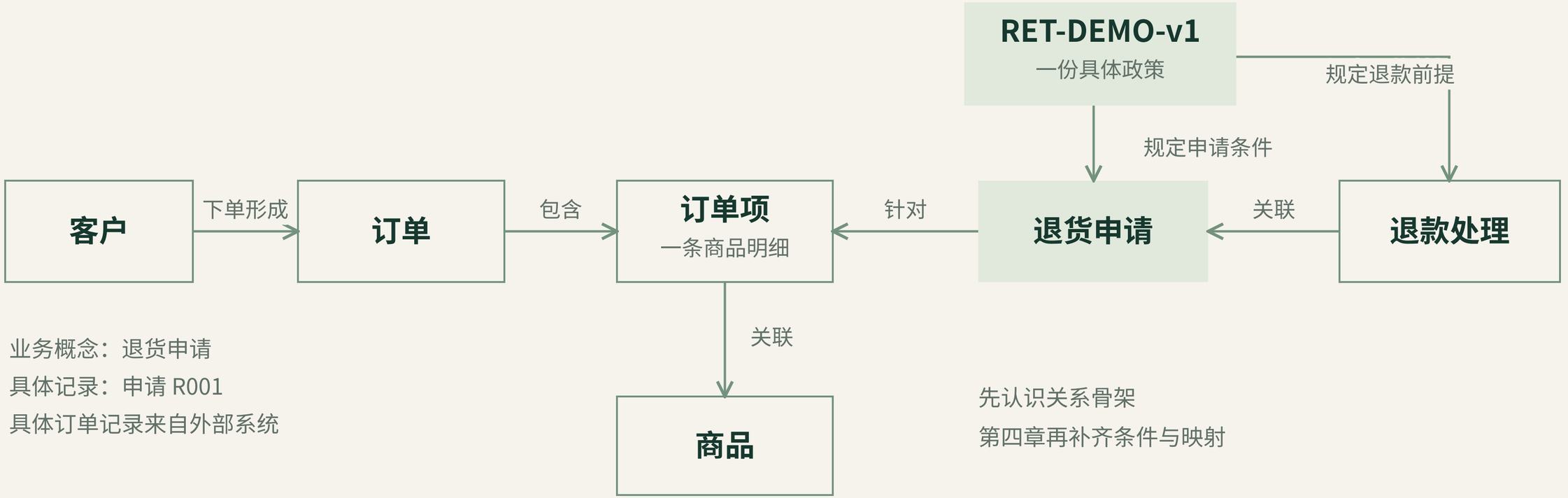
可共同使用、机器可处理的模型。



专业表述：显式的领域概念化表达。定义、身份与关系共同支持复用。

用本体描述退货业务

先认识对象和关系骨架，稍后补齐条件、事实来源与动作前提

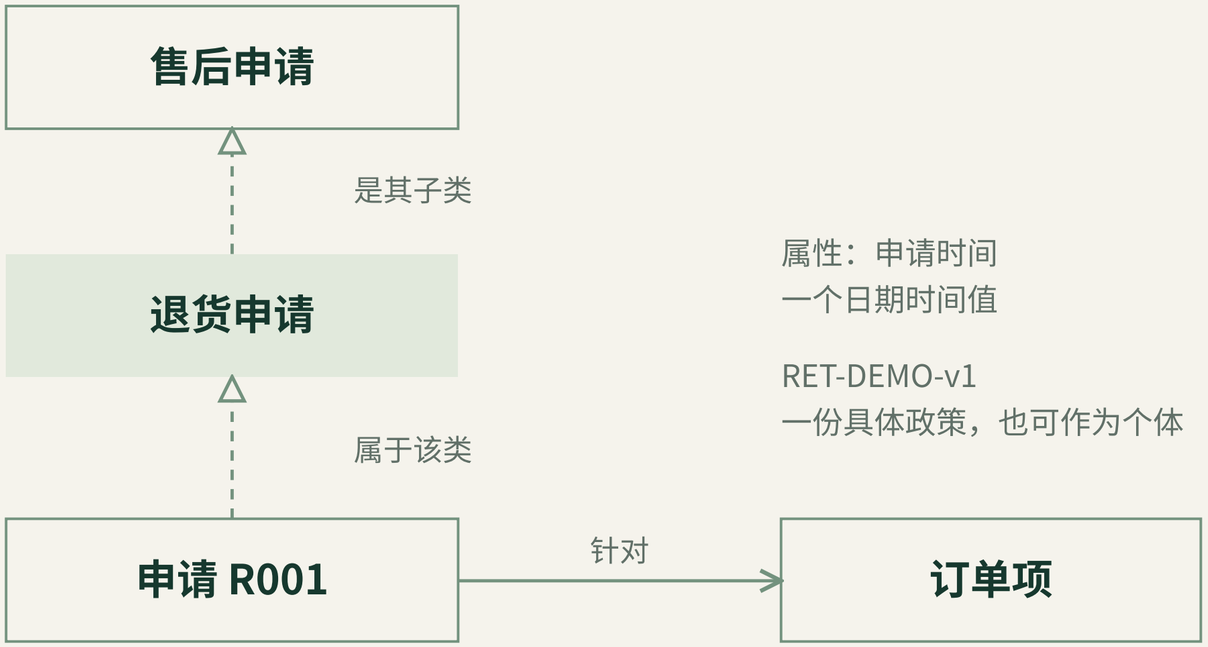


订单项是一条商品明细；“退货申请”这个概念与“申请 R001”这条记录分别表达

业务模型示意

本体中的基本表达

在同一小段业务中，读懂类别、个体、属性、关系和分类



类描述一类对象

个体指一个具体资源

属性连接值

关系连接资源

虚线表示分类／类型关系
实线表示业务关系
业务说明同时保留范围、例外与未知

名字相似不等于含义相同；同一资源需要稳定身份

本体、知识图谱与数据库的分工

同一业务，在定义、知识连接和记录存储三个层面表达

本体：约定含义与关系

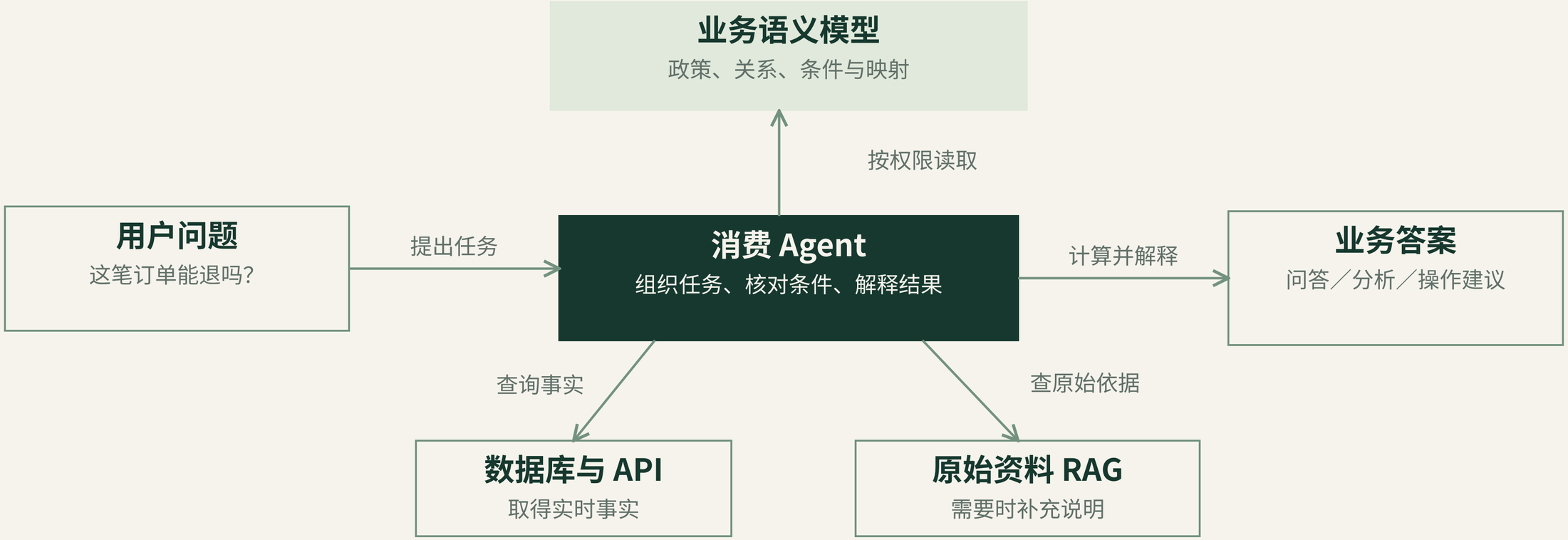
定义对象与可用关系



本项目将订单等业务行保留在外部数据库，平台保存业务语义及映射

本体怎样帮助 Agent 工作

相关定义、关系和必要说明，合起来形成“语义上下文”



本体提供依据，Agent 组织任务与解释，外部系统提供事实并执行操作

业务含义怎样表达与查询

RDF、RDFS、OWL 与 SPARQL 各有职责

RDF 以“主体、谓词、客体”表达陈述



IRI：为资源提供可稳定引用的身份

Turtle：RDF 的一种书写格式

教学词汇示意，不代表专用业务规则执行合同

RDFS　基础类型与层级

`rdf:type`　属于某类

`rdfs:subClassOf`　子类

OWL　更丰富的形式语义

表达形式化公理及其含义

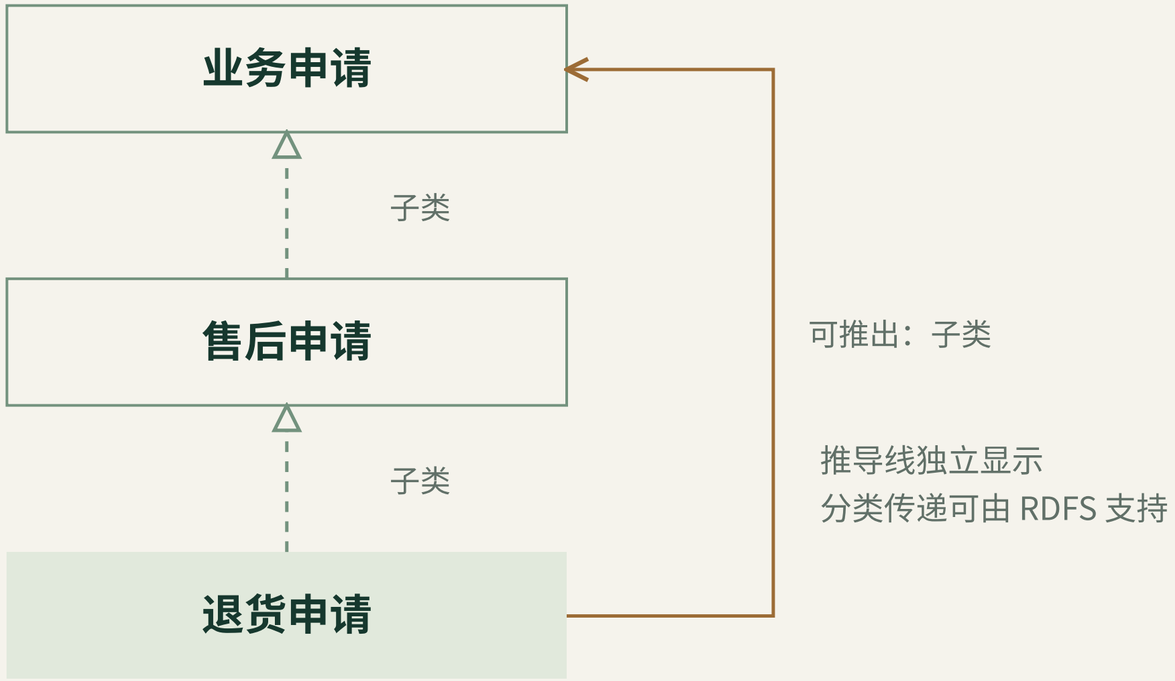
SPARQL　查询 RDF 图

找到符合图关系的资源与值

能机器读取业务说明，不等于其中所有条件已经形式化或能够自动执行

形式推理的作用与边界

由明确的形式前提推出隐含结论，与退款资格计算分别理解



原理示意

显式前提

形式规则

隐含结论

时间比较与金额计算需计算能力。
业务流程由对应系统执行。
缺少事实，不自动意味着事实为假。

OntoWeave 当前不提供自动推理运行

分类推导不能单独证明某张订单可以退款

Ontology 的思想来源与工程应用

相关思想、表示方法与产品实践，有联系，也有各自的边界



哲学 ontology 关注存在。

工程 ontology 将领域概念显式表达。

Palantir 是重要产品应用案例，并非 ontology 的发明者。

Foundry 产品体系不等同于 OWL 本体或 OntoWeave。

“本体驱动 Agent”是利用显式业务语义的可选架构思路

这是一条理解联系的线索，不代表唯一或必经的技术演进路线

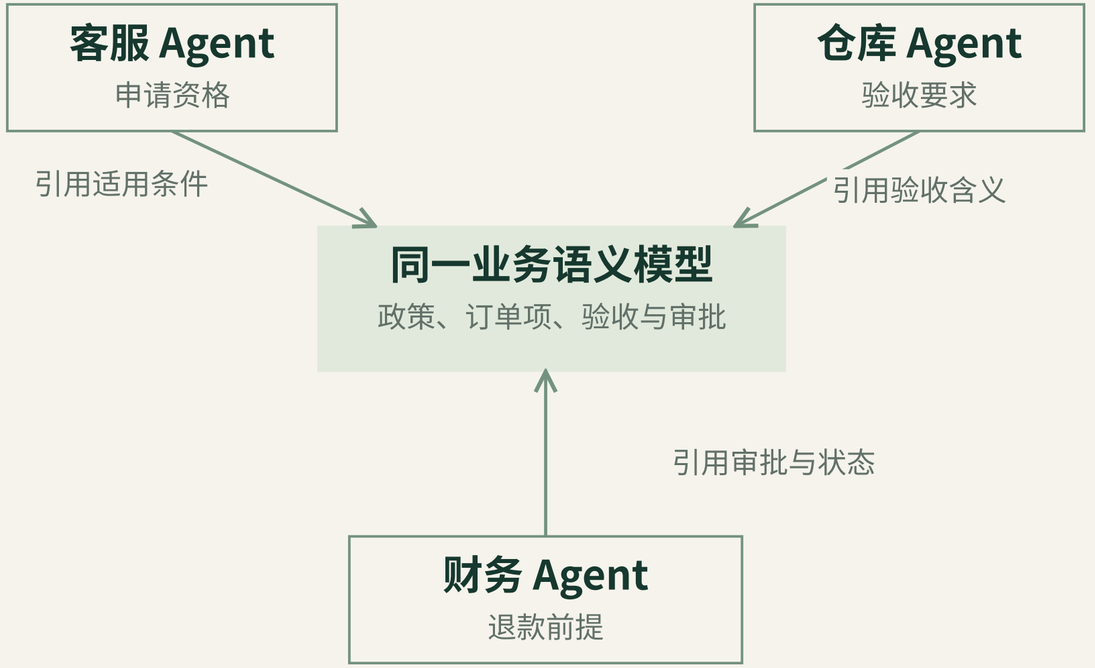
本体的优势与适用条件

用“为什么还不能退款”连接客服、仓库与财务的任务

共享口径已有轻量方案

统一文档、业务词典、
共享配置与规则服务，
都能帮助多个应用复用定义。

固定、简单的任务，可以继续使用更轻的方案。



值得建模的地方，在于跨对象、跨任务、跨系统的显式关联能否改善实际复用

第 3 章

OntoWeave 与建模方法

03

如何把业务材料变成可用的模型，
并让 Agent 可靠地取得它。

系统分工 ／ 建模方法 ／ 自动建模 ／ 读取验证

OntoWeave 的定位与责任边界

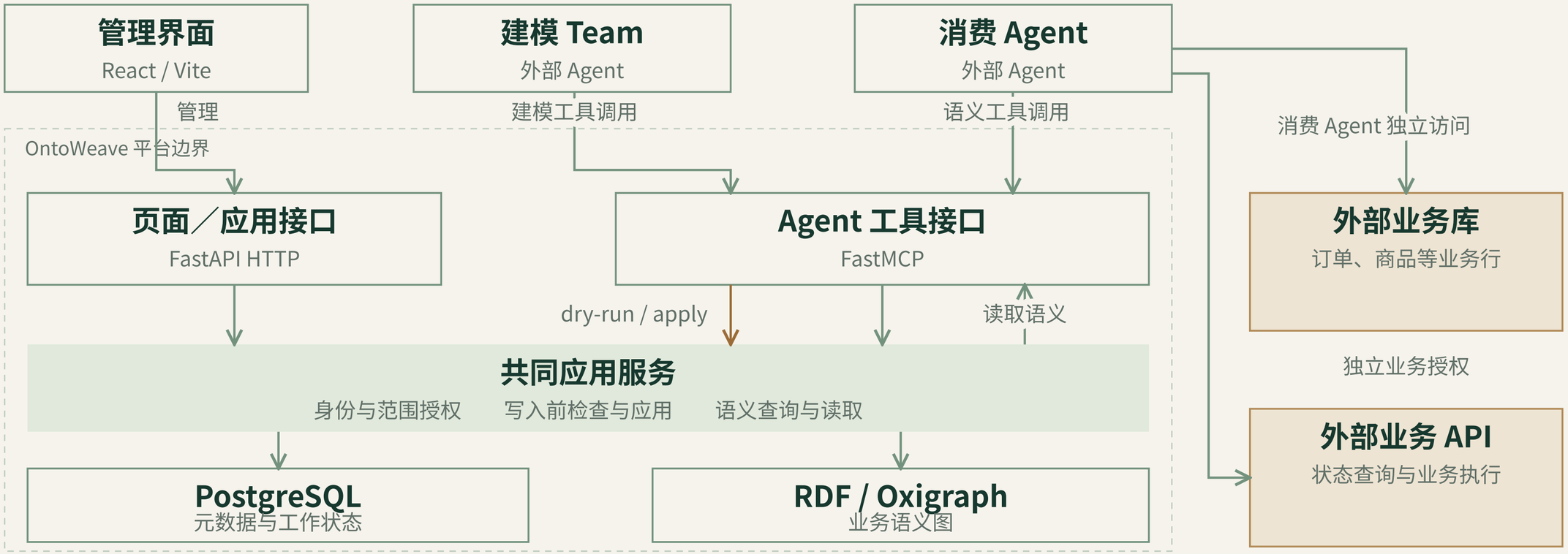
平台保存与提供业务语义，业务判断和业务执行各有负责人



平台检查可执行性与资源完整性，业务含义仍由建模与独立验证确认

系统架构

两个接口入口复用应用服务，外部业务库与业务 API 保持独立



业务数据与执行仍在平台外；dry-run 检查待写内容，apply 应用已确认的写入

建模方法论：先完成一条可验证的业务链

从问题确定范围，再逐步扩展，保留项目级共享定义与依赖

01

典型业务问题

“能申请，是否就能退款？”
这类问题称为能力问题 CQ

02

逻辑切片

按业务目标、生命周期与依赖
分步分析和验证

03

业务语义模型

先完成最小完整链
用七个视角检查覆盖

04

实际读取与回答

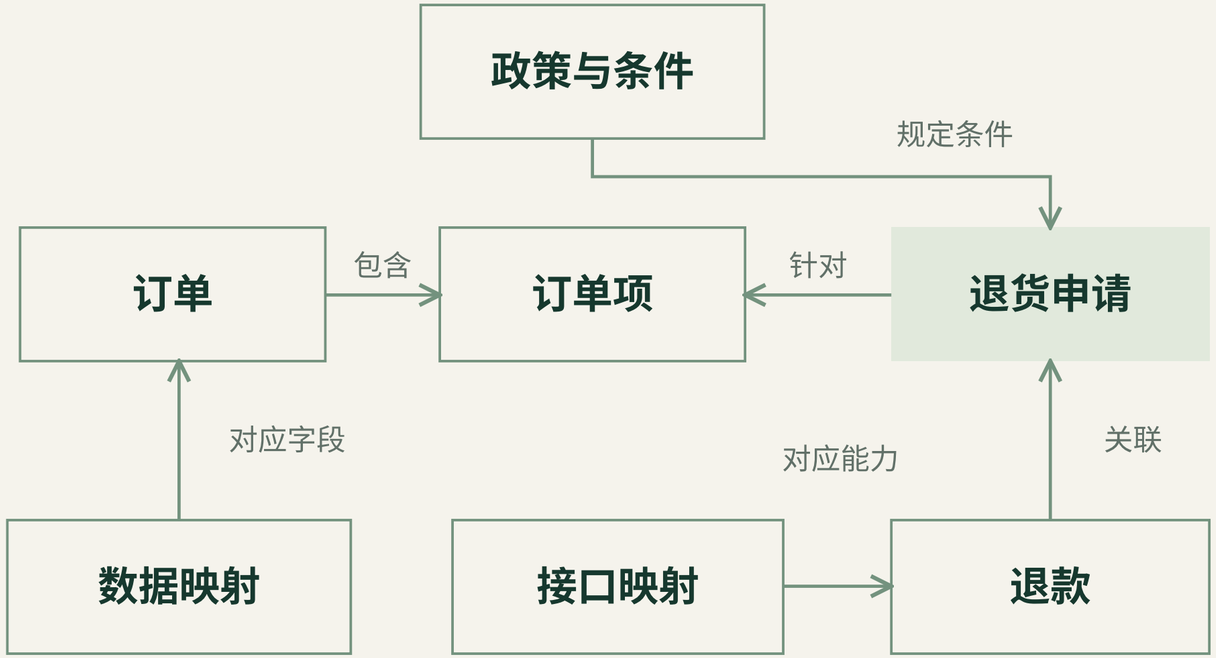
检查消费者是否正确使用
再扩展相邻切片

项目级始终保留：**共享术语、跨切片依赖、数据与接口映射、跨切片问题。**
读取和回答中的缺口，返回业务澄清与语义修订。

切片是组织分析与验证的方式；不按表数量直接决定有几个本体

七个视角检查同一业务模型

同一模型，从含义、参与者、条件与系统连接等角度检查覆盖



对象：业务中讨论什么

订单、订单项、商品与退货申请。确认每个对象的含义、身份和必要关系。

各视角记录：
已确认／待澄清／本范围不适用。

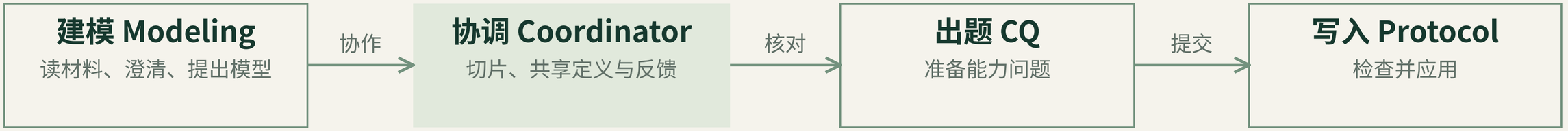
行为：提交、审核、退款。规则：条件、范围与未知。角色：客户、客服、仓库。流程：申请至退款。映射：数据字段与外部能力。

七个视角检查同一业务，不要求七个 Ontology 或七个 Agent

Agent Team 的自动建模方式

生产协作与独立验收分别承担职责，业务歧义仍需业务确认

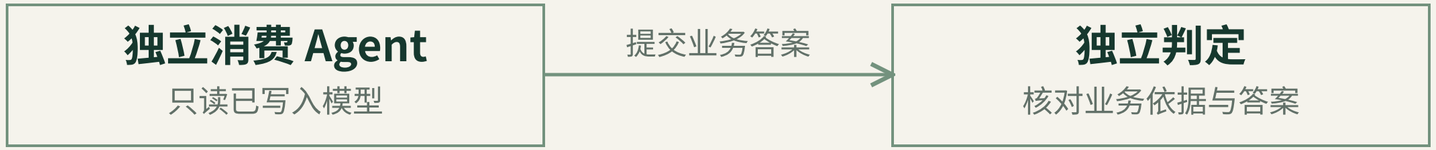
生产协作



宿主监督 Delivery：授权、进度、交付与失败跟进

独立验收

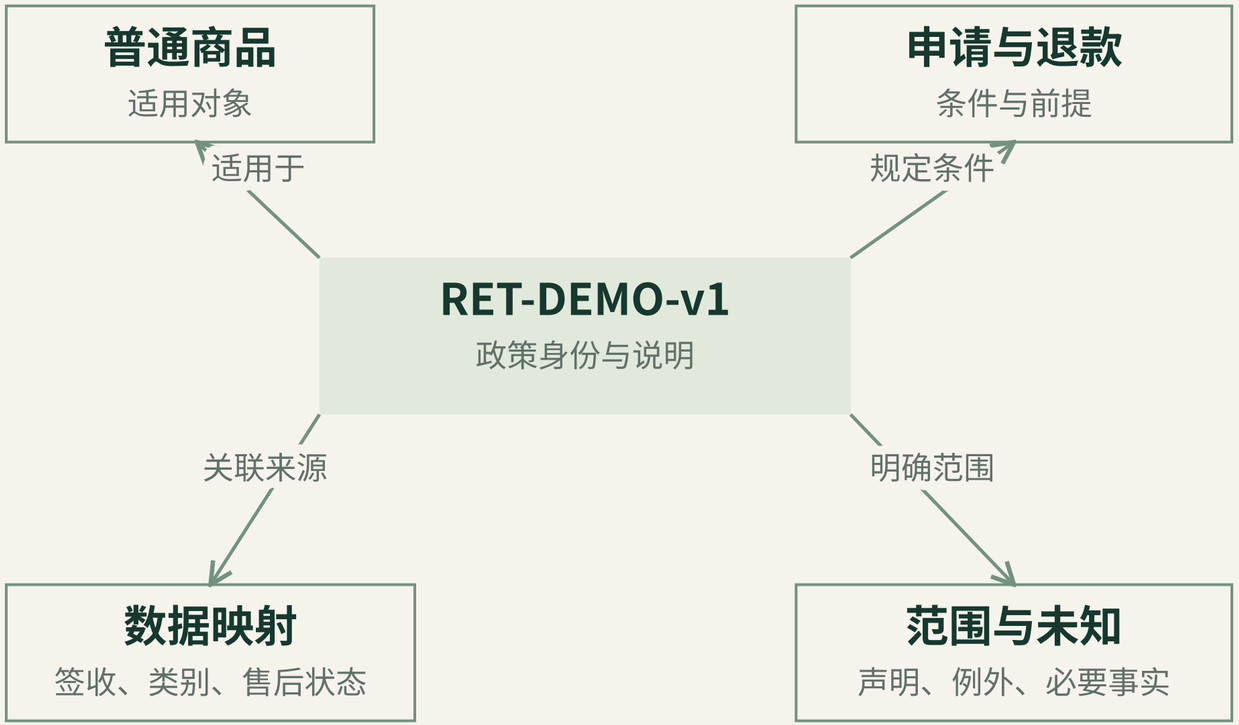
公开反馈可用于修订
隐藏答案保持隔离



自动整理、建模和调用不替代业务确认；成功写入与业务语义通过分别证明

消费 Agent 实际需要读到什么

把“语义上下文”展开为能支撑业务判断的内容



RET-DEMO-v1

适用对象	普通商品，单件整件退货
申请条件	客户声明未使用 0 ⩽ 间隔 ⩽ 168 小时
退款前提	退回验收通过、客服批准
事实来源	`orders.signed_at` 客户输入、外部售后接口
范围与状态	按权限读取，检查分页／截断 缺必要信息时补读或询问

按权限读到必需内容，并检查分页与截断；读取状态不保证答案正确

虚构内容与展示形式，非真实接口响应

当前能力与验证边界

截至 2026-10-14，按需求及保留交付记录陈述

已实现能力

当前需求与交付范围

多视角语义表达、共享定义、确定性写入、受治理读取，
以及数据与接口映射表达。**工程检查通过，不等于业务效果已证明。**

已有有界证据

2026-10-06 · 工业服务四切片

冻结题集独立验收：**公开 18/18、隐藏 8/8**。
仅覆盖当时获准模型与题集，不能作为当前通用准确率。

仍待验证范围

10 月 13—14 日后续场景

后续运行存在未通过或未完成验收，R4.0-009 整体仍为 **PARTIAL**。
全范围、SQL/API 联合验收、跨场景重复性与产品化尚未完成。

当前不提供 OWL 自动推理、SHACL 专用校验、规则脚本或业务流程执行引擎。

历史有界通过保留原义，当前部分验证保留边界；本次没有重跑平台验收

已有记录摘要，本次未重新验收

第 4 章

电商退货与退款模拟案例

04

一家小型电商，希望让客服 Agent 判断退货资格，
并说明退款的下一步。

公开材料 ／ 确认口径 ／ 建立语义 ／ 取得事实 ／ 解释与验证

模拟业务背景与输入材料

A 店 · 单件商品整件退货 · 商品款整额退款

政策与业务确认
术语解释
数据库字段与编码说明
售后接口合同
客户本次声明

不涉及运费、促销分摊、部分退货、
跨境业务或法律解释。

三个对照订单

A101	10-01 10:00 签收 普通商品，声明未使用
A102	09-27 10:00 签收 普通商品，声明未使用
A103	签收时刻缺失 普通商品，声明未使用

拟申请／判断时点：**2026-10-06 10:00**
全部时间为 UTC+08:00

先公开范围、政策与事实来源，后续每个判断才能逐项核对

全部材料与订单均为虚构

建模前的业务确认

字段名和样例值是映射线索，业务确认决定政策含义

01

“七天内”

需要问：从哪一刻起算？
按日历日还是连续小时？

确认：提交减签收，
0—168 小时，包含等号。

02

“没使用就能退”

需要问：申请时怎样确认？
定制商品是否适用？

确认：普通商品，
申请时采用客户未使用声明。

03

“申请后退款”

需要问：还有哪些前提？
信息缺失怎样处理？

确认：验收通过、客服批准。
缺必要事实时保留未知。

客户声明与实际验收分别记录；申请受理、具备退款条件和退款完成分别判断

业务切片与共享定义

把 P21 的方法具体化为三段范围，逐步完成同一业务链

01

退货资格

确认适用商品、声明与时间窗。

首先回答：

是否允许提出申请?

02

退货处理

表达申请、退回与验收状态。

继续回答：

商品实际状态如何确认?

03

退款处理

表达审批、退款前提与接口。

跨环节回答：

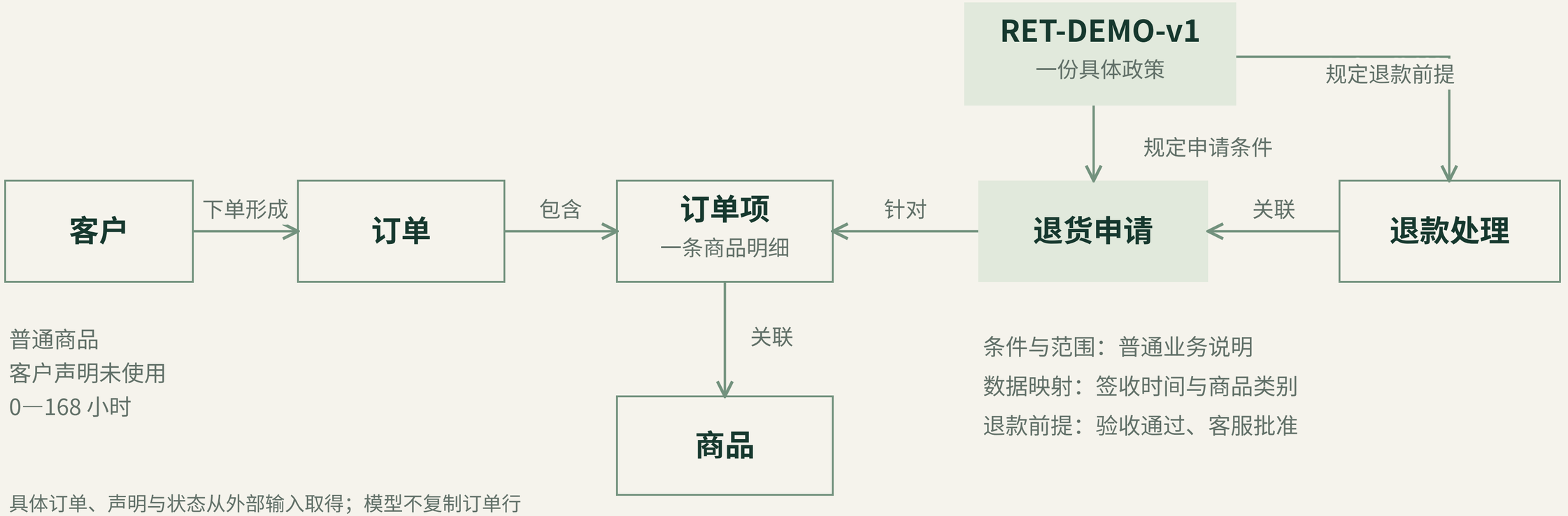
何时具备退款条件?

> 共同引用：**客户、订单项、商品、政策、状态含义，以及必要数据与接口映射。**
> 首条链也需保留“可以申请与退款完成是不同阶段”这一共同语义。

三个逻辑切片是分析组织方式，不预设三套本体或三个 Agent

退货与退款的语义模型

沿用 P10 的骨架，补上已经确认的条件、前提与映射



关系用于定位相关知识，普通业务说明保留条件、范围、例外和未知

语义怎样连接 SQL 与 API

先按获准语义取得事实，再由外部系统控制业务执行

查询事实

签收时刻 → `orders.signed_at`

商品类别 → `products.category`

订单项通过 `order_id` 与 `product_id`
连接订单和商品。语义映射约束 SQL。

> 声明来自**客户本次输入**。
> 验收、审批与退款状态来自**外部售后查询接口**。

执行动作

提交申请 → `POST /returns`

执行退款 → `POST /refunds`

外部系统检查条件、业务权限与防重复。
语义知识中的前提，不授予操作权限。

> `ACCEPTED` 只表示退款请求受理。
> `SUCCEEDED` 才表示系统确认完成。

SQL、时间计算工具和 API 都是教学设计，本次没有接入或调用退款系统

案例一：能申请，为什么不能马上退款

A101 · RET-DEMO-v1 · 2026-10-06 10:00+08:00 的模拟判断

RET-DEMO-v1

适用对象	普通商品，单件整件退货
申请条件	客户声明未使用 0 ⩽ 间隔 ⩽ 168 小时
退款前提	退回验收通过、客服批准
事实来源	`orders.signed_at` 客户输入、外部售后接口

取得具体事实

订单库	10-01 10:00 签收 普通商品
客户输入	声明未使用
计算工具	间隔 **120 小时** 0 ⩽ 120 ⩽ 168
售后接口	验收尚未开始 审批尚未申请 无退款记录

按当前政策和您的未使用声明，可以申请退货。

实际商品状态仍待验收。

当前尚未验收，也尚未获客服批准，**不能立即退款。**

下一步：如决定申请，由业务系统在实际提交时重检条件与权限。

可申请、已提交、具备退款条件、退款完成是不同阶段

案例二：超时与信息不足

相同政策，对明确不满足与无法判断分别回答

A102 · 超时

216 小时

09-27 10:00 签收，10-06 10:00 拟申请。
普通商品，客户声明未使用。

超过 168 小时，
不符合本例政策时间条件。

解释明确不满足的条件，
不编造退款阶段事实。

A103 · 必要信息缺失

未知

签收时刻缺失，无法计算间隔。
普通商品，客户声明未使用。

暂无法判定时间条件，
需要补充签收时刻。

保留未知并提出补问，
不将未知改写成允许或拒绝。

明确不满足的条件可以支持否定结论；必要条件尚未确定时保留相应未知

怎样验证建模效果

独立检查含义、实际读取与业务回答，避免把写入成功当作质量证明

资料依据

先确定能力问题、适用范围
和可核对的判定依据

模型表达

概念、关系、条件、例外
与未知是否忠于资料

实际检索

在真实授权接口读取
必需内容是否取得

业务答案

独立消费 Agent 回答
独立判定核对依据

同条件比较

RAG 与本体辅助方案共用相同资料、澄清结果、政策版本、数据快照、题目、工具与权限。
尽量固定语言模型及调用条件，改变知识组织与读取方式。

A101：可申请，不能立即退款　A102：超时　A103：补问　168 小时：含等号

分别记录错误允许、错误拒绝、应补问却强答与合理未知，以及建模、维护、确认和运行成本。

显式语义连接依据，外部事实支持结论，业务系统控制执行，独立验证检查整条链

FAQ：什么时候值得使用本体

先看当前任务与已有方案，再决定是否增加显式语义建模

已经有 RAG，还需要本体吗？

文档检索已能稳定回答时，可以继续使用。跨对象、跨任务的关系需要明确复用时，可试用本体并做同条件比较。

直接把业务规则写进提示词不行吗？

小范围任务可以。文档、词典和规则服务也能共享口径。显式关联带来的复用收益值得投入时，再进一步建模。

本体会替代数据库或业务系统吗？

业务记录与操作仍留在原系统。本体提供业务含义、关系、条件及系统映射。

所有业务都要先建本体吗？

优先选择含义歧义多、跨系统关系多、同一定义被反复使用的场景。简单固定任务可以采用更轻的方案。

回看方法局限 P07　回看系统分工 P12　回看适用条件 P17

从一个有代表性的跨环节问题试起，比较正确性与实际投入

FAQ：怎么建，哪些工作需要人参与

从一条有界业务链开始，按实际依赖安排分工

必须先学会 RDF 和 OWL 吗?

业务人员先确认概念与口径，技术人员选择表达及查询方式。理解本体不要求先写 OWL，当前项目也不要求自动推理。

七个视角意味着七个本体或七个 Agent 吗?

它们是检查同一业务的不同角度。切片、存储和人员分工按实际依赖安排。

Agent 能全自动把模型建好吗?

可以自动整理资料、提出模型并调用平台。资料缺失、口径冲突与授权仍需确认，质量由独立验证判断。

是否需要一次建完整个企业?

先完成一条有代表性、可验证的业务链，再维护共享定义，并逐步扩展相邻切片。

建模方法 P21　七个视角 P22　Team 分工 P23

自动化承担整理与调用，业务人员确认含义，独立验证判断是否可用

FAQ：怎样保证质量并持续维护

定义、事实、检索和使用都需要检查；政策变更还需外部系统对齐

有了本体就不会产生幻觉了吗？

源材料、模型、检索与 Agent 使用都可能出错。需要逐环节验证，也允许合理回答未知。

业务规则变了，是否要全部重建？

业务负责人确认新口径、生效时间和范围，再修订相关定义及映射，检查受影响问题。修改范围由实际依赖决定。

怎样评估成本与收益？

同资料、同澄清、同工具比较正确性和完整性，分别记录建模、维护、人工确认与调用成本。

本体权限能保护数据库和退款吗？

语义权限控制语义知识。数据库行列访问与退款操作权限由外部系统执行，必须分别验证。

政策变更示例

168 → 240 小时

自 **2026-11-01 00:00+08:00**
起提交的申请采用 v2。
此前提交的仍按 v1。

业务负责人确认
维护者修订语义
系统负责人对齐执行
独立验证新旧边界

独立教学变更，不改变前三例。
不代表自动影响分析或任意历史版本读取功能。

结合你的业务继续讨论：哪些含义、关系与判断，最需要共同引用？